SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 40%

QUESTIONS: 10 Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I ANISH K/192521288 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

1.An airport has five check-in counters serving passengers. Each passenger arrival is considered a process with varying arrival and service times. During peak hours, passengers with special assistance are given priority. Analyze the scheduling using FCFS and Priority Scheduling. Determine the waiting time and turnaround time for each passenger and identify which algorithm provides better service efficiency.

Introduction:

An airport has multiple check-in counters where passengers arrive continuously for ticket verification and baggage check-in. In Operating Systems, each passenger is considered a process, and each check-in counter acts as the CPU. The operating system schedules passengers using different scheduling algorithms to reduce waiting time and improve service efficiency. During peak hours, passengers requiring special assistance, such as elderly people, pregnant women, disabled passengers, or medical emergencies, are given higher priority.

FCFS (First Come First Serve):

FCFS is the simplest CPU scheduling algorithm. Passengers are served in the exact order of their arrival at the check-in counter.

Working:

- The passenger who arrives first is served first.
- Once a passenger starts check-in, the process continues until completion.
- No passenger can interrupt the current check-in process.
- It is a non-preemptive scheduling algorithm.

Advantages:

- Easy to understand and implement.
- Fair because passengers are served in arrival order.
- No starvation occurs.
- Suitable during normal airport operations.

Disadvantages:

- Emergency passengers may have to wait.
- Long service times increase the waiting time for other passengers.
- Suffers from the convoy effect, where one slow passenger delays everyone behind**.**

Priority Scheduling:

Priority Scheduling assigns a priority level to every passenger. Passengers requiring special assistance receive higher priority than regular passengers.

Working:

- Each passenger is assigned a priority.
- The passenger with the highest priority is served first.
- If two passengers have the same priority, FCFS is followed.
- It can be implemented as preemptive or non-preemptive scheduling**.**

Advantages:

- Reduces waiting time for special assistance passengers.
- Improves passenger safety and satisfaction.
- Efficient during peak hours.
- Ensures urgent cases are handled immediately.

Disadvantages:

- Regular passengers may wait longer.
- Continuous arrival of high-priority passengers may cause starvation.
- More complex to implement than FCFS.

Example:
Suppose five passengers arrive at the airport.

Passenger	Arrival Time (min)	Service Time (min)	Priority
P1 (Regular)	0	5	3
P2 (Regular)	1	4	3
P3 (Special Assistance)	2	3	1
P4 (Regular)	3	2	3
P5 (Regular)	4	4	3

Priority: 1 = Highest, 3 = Lowest
FCFS Scheduling
Execution Order:
P1 → P2 → P3 → P4 → P5
Even though P3 is a special assistance passenger, they must wait until P1 and P2 complete their check-in.
Priority Scheduling
Execution Order:
P1 → P3 → P2 → P4 → P5
After P1 completes, P3 is immediately served because of the highest priority, reducing the waiting time for the passenger who needs urgent assistance.

Comparison:

FCFS	Priority Scheduling
Based on arrival time	Based on passenger priority
Simple implementation	Slightly complex implementation
Fair to all passengers	Fair to urgent passengers
No starvation	Starvation may occur
Higher waiting time for emergency passengers	Lower waiting time for emergency passengers
Suitable during normal hours	Suitable during peak hours and emergencies

Service Efficiency Analysis:

- FCFS ensures fairness because every passenger is served according to arrival time, but it cannot handle emergency situations efficiently.
- Priority Scheduling significantly reduces the waiting time for passengers requiring immediate assistance.
- During peak hours, Priority Scheduling improves airport operations by serving urgent passengers quickly.
- To prevent starvation of regular passengers, the Aging technique can be applied, where the priority of waiting passengers gradually increases over time.

Conclusion:

For airport check-in counters, Priority Scheduling provides better service efficiency because it minimizes the waiting time for passengers requiring special assistance while ensuring smooth airport operations. FCFS is suitable during normal operating hours due to its simplicity and fairness, whereas Priority Scheduling is the preferred choice during peak hours and emergency situations. Therefore, a combination of FCFS and Priority Scheduling can provide both fairness and efficiency in airport passenger management.

---

2. A hospital emergency room receives patients continuously. Critical patients must be treated immediately, while non-critical patients wait in a queue.Model this as a process scheduling problem using Priority Scheduling and Round Robin (time quantum = 5 minutes). Compare the response time for critical and non-critical patients.

Introduction:

A hospital emergency room (ER) receives patients continuously with different levels of medical urgency. In Operating Systems, each patient is considered a process, and the doctor or medical team acts as the CPU. The operating system schedules patients using CPU scheduling algorithms to provide timely treatment. Critical patients such as accident victims or heart attack patients require immediate attention, while non-critical patients can wait. Therefore, Priority Scheduling and Round Robin Scheduling are commonly used to model this scenario.

Priority Scheduling:

Priority Scheduling is a CPU scheduling algorithm in which every process is assigned a priority. The process with the highest priority is executed first.

Working:

- Each patient is assigned a priority based on the severity of the illness.
- Critical patients receive the highest priority.
- Doctors immediately begin treatment for high-priority patients.
- Patients with the same priority are treated using FCFS.
- It can be implemented as preemptive or non-preemptive scheduling.

Advantages:

- Immediate treatment for emergency patients.
- Very low response time for critical cases.
- Increases the chances of saving lives.
- Efficient use of hospital resources.
- Suitable for emergency departments.

Disadvantages:

- Non-critical patients may wait for a long time.
- Continuous arrival of emergency patients may cause starvation.
- Requires proper medical assessment to assign priorities.

Round Robin Scheduling:

Round Robin is a preemptive scheduling algorithm in which each process receives an equal amount of CPU time called the time quantum. Here, the time quantum is 5 minutes.

Working:

- Patients are placed in a queue.
- Each patient receives treatment for a maximum of 5 minutes.
- If treatment is not completed, the patient moves to the end of the queue.
- The doctor continues treating patients one by one until all treatments are completed.

Advantages:

- Equal treatment opportunity for all patients.
- Prevents starvation.
- Fair scheduling.
- Suitable for outpatient departments (OPD).
- Easy to implement.

Disadvantages:

- Critical patients may not receive immediate treatment.
- Frequent switching between patients reduces efficiency.
- Not suitable for life-threatening emergencies.

Example:

Suppose five patients arrive at the emergency room.

Patient	Condition	Arrival Time	Treatment Time	Priority
P1	Fever	0 min	8 min	3
P2	Heart Attack	1 min	10 min	1
P3	Fracture	2 min	6 min	2
P4	Cold	3 min	5 min	4
P5	Accident	4 min	7 min	1

Priority: 1 = Highest, 4 = Lowest

Priority Scheduling

Execution Order

P2 → P5 → P3 → P1 → P4

- Heart attack (P2) is treated first.
- Accident patient (P5) is treated next.
- Non-critical patients wait until emergency patients are treated.

Observation:

- Critical patients receive immediate medical attention.
- Waiting time for emergency patients is very low.
- Non-critical patients experience longer waiting times.

Round Robin Scheduling (Time Quantum = 5 minutes)

Execution Order

P1 → P2 → P3 → P4 → P5 → P1 → P2 → P3 → P5

Each patient receives **5 minutes** of treatment before the doctor moves to the next patient.

Observation:

- Every patient gets an equal opportunity for treatment.
- Critical patients are not treated immediately.

- Waiting time is distributed more fairly among all patients.

Comparison of Response Time:
Priority Scheduling:
- Critical patients receive treatment immediately.
- Response time for emergency patients is very low.
- Non-critical patients may have to wait longer.

Round Robin Scheduling:
- Every patient receives equal treatment time.
- Critical patients may wait until their turn.
- Better fairness but slower response for emergencies.

Comparison Table:

Feature	**Priority Scheduling**	**Round Robin Scheduling**
Scheduling Basis	Patient Priority	Time Quantum (5 min)
Type	Preemptive / Non-Preemptive	Preemptive
Response Time for Critical Patients	Very Low	Moderate
Response Time for Non-Critical Patients	High	Low and Fair
Fairness	Moderate	High
Starvation	Possible	Not Possible
Context Switching	Low	High
Best Used In	Emergency Room	General OPD

Analysis:
- In an emergency room, saving lives is the highest priority.
- Priority Scheduling ensures that patients with life-threatening conditions are treated immediately, reducing response time and improving survival chances.
- Round Robin is fair because every patient receives equal treatment time, but it is not ideal for emergencies as critical patients may have to wait.
- To avoid starvation in Priority Scheduling, the **Aging** technique can be used, where the priority of waiting non-critical patients gradually increases over time.

Conclusion:

Priority Scheduling is the most suitable scheduling algorithm for a hospital emergency room because it provides immediate treatment to critical patients, reducing response time and improving patient safety. Round Robin Scheduling is better suited for outpatient departments where fairness among patients is more important than urgency. Therefore, hospitals often use Priority Scheduling with Aging to achieve both efficiency and fairness**.**

3.A cloud service provider receives ten computational tasks from clients. Each task requires different CPU burst times and arrives at different intervals.Apply FCFS, SJF, and Round Robin scheduling to determine the average waiting time and CPU utilization. Recommend the best scheduling algorithm for the cloud environment.

INTRODUCTION:

A cloud service provider receives multiple computational tasks from different clients. Each task is treated as a process and requires different CPU burst times. The operating system schedules these tasks using CPU scheduling algorithms to improve system performance, reduce waiting time, and maximize CPU utilization. The commonly used scheduling algorithms are First Come First Serve (FCFS), Shortest Job First (SJF), and Round Robin (RR).

FIRST COME FIRST SERVE (FCFS):

FCFS is the simplest CPU scheduling algorithm. Processes are executed in the order in which they arrive.

WORKING:

- Tasks are stored in the ready queue according to arrival time.
- The task that arrives first gets the CPU first.
- Once a task starts execution, it continues until completion.
- It is a non-preemptive scheduling algorithm.

ADVANTAGES:

- Simple and easy to implement.
- Fair because tasks are executed in arrival order.
- No starvation occurs.
- Suitable for batch processing systems.

DISADVANTAGES:

- Long tasks delay shorter tasks.
- Average waiting time is high.
- Poor response time.
- Suffers from the convoy effect.

SHORTEST JOB FIRST (SJF):

SJF selects the process having the smallest CPU burst time for execution.

WORKING:

- Compare the burst time of all available tasks.
- Execute the task with the shortest burst time first.
- Repeat until all tasks are completed.
- It may be preemptive or non-preemptive.

ADVANTAGES:

- Produces minimum average waiting time.
- Improves turnaround time.
- Better CPU utilization.
- Increases overall system performance.

DISADVANTAGES:

- Burst time must be known in advance.
- Long processes may suffer starvation.
- Difficult to implement in dynamic environments.

ROUND ROBIN (RR):

Round Robin Scheduling allocates CPU time equally to every process using a fixed time quantum.

WORKING:

- Every task receives a fixed amount of CPU time.
- If the task is not completed, it returns to the end of the ready queue.
- The next task receives CPU time.
- The process continues until all tasks finish execution.

ADVANTAGES:

- Fair scheduling.
- No starvation.
- Good response time.
- Suitable for time-sharing systems.

DISADVANTAGES:

- Frequent context switching.
- Larger overhead.
- Performance depends on the selected time quantum.

EXAMPLE:

Suppose a cloud server receives the following tasks.

TASK	ARRIVAL TIME (ms)	BURST TIME (ms)
T1	0	8
T2	1	4
T3	2	9
T4	3	5
T5	4	2

FCFS EXECUTION ORDER:

T1 → T2 → T3 → T4 → T5

OBSERVATION:

- Tasks are executed according to arrival time.
- Short tasks such as **T5** wait behind longer tasks.
- Average waiting time becomes higher.

SJF EXECUTION ORDER:

T5 → T2 → T4 → T1 → T3

OBSERVATION:

- Shorter tasks finish quickly.
- Average waiting time is reduced.
- CPU utilization improves because more tasks complete in less time.

ROUND ROBIN EXECUTION ORDER (TIME QUANTUM = 4 ms):

T1 → T2 → T3 → T4 → T5 → T1 → T3 → T4 → T3

OBSERVATION:

- Every task gets equal CPU time.
- No task waits indefinitely.

- Suitable when multiple users access cloud services simultaneously.

COMPARISON:

FEATURE	FCFS	SJF	ROUND ROBIN
Scheduling Basis	Arrival Time	Shortest Burst Time	Time Quantum
Type	Non-Preemptive	Preemptive/Non-Preemptive	Preemptive
Average Waiting Time	High	Lowest	Moderate
CPU Utilization	Moderate	High	High
Fairness	Good	Moderate	Excellent
Starvation	No	Possible	No
Context Switching	Low	Low	High
Best Application	Batch Systems	Cloud Computing	Interactive Systems

ANALYSIS:

- FCFS is simple but increases waiting time when long tasks arrive first.
- SJF minimizes waiting time and turnaround time by executing short tasks first.
- Round Robin provides equal CPU time to every task and improves response time but introduces additional context switching.
- In cloud environments, users expect quick execution and efficient resource utilization. Therefore, SJF generally offers better overall performance, while Round Robin is preferred for interactive cloud services where fairness is important.

RECOMMENDATION:

For a cloud service provider, Shortest Job First (SJF) is the most suitable scheduling algorithm because it minimizes average waiting time, improves turnaround time, and increases CPU utilization. If fairness among multiple users is required, Round Robin can also be used.

CONCLUSION:

Among the three scheduling algorithms, SJF provides the best performance for cloud environments because it executes shorter tasks first, reducing waiting time and improving CPU utilization. FCFS is simple but less efficient, while Round Robin is ideal for time-sharing systems where every client should receive an equal share of CPU time.

4.A smart city uses sensors to detect vehicle density at an intersection. Each traffic lane is represented as a process competing for signal access.Design a scheduling strategy that minimizes vehicle waiting time. Compare the effectiveness of Round Robin and Priority Scheduling for emergency vehicles.

INTRODUCTION:

A smart city uses intelligent traffic management systems to control traffic flow at road intersections. In this scenario, each traffic lane is considered a **process**, and the traffic signal acts as the **CPU** that allocates green signal time. The operating system schedules each lane to minimize vehicle waiting time and avoid congestion. During emergencies, vehicles such as ambulances, fire engines, and police vehicles must be given immediate access. Therefore, **Round Robin Scheduling** and **Priority Scheduling** can be used to manage traffic efficiently.

ROUND ROBIN SCHEDULING:

Round Robin Scheduling allocates an equal amount of green signal time to each traffic lane in a cyclic order. Every lane gets a fixed time quantum before the signal switches to the next lane.

WORKING:

- Each traffic lane is placed in a queue.
- Every lane receives a fixed green signal duration.
- After the allotted time expires, the signal moves to the next lane.
- The cycle continues until traffic from all lanes is cleared.

ADVANTAGES:

- Ensures fairness by providing equal signal time to every lane.
- Prevents starvation of any traffic lane.
- Reduces traffic congestion during normal traffic conditions.
- Simple to implement in automated traffic systems.
- Suitable for roads with similar traffic density.

DISADVANTAGES:

- Emergency vehicles may have to wait until their turn.
- Equal signal time may not suit roads with heavy traffic.
- Frequent signal switching may reduce efficiency.

PRIORITY SCHEDULING:

Priority Scheduling assigns higher priority to lanes based on traffic conditions or vehicle importance. Emergency vehicle lanes receive the highest priority and are allowed to pass first.

WORKING:

- Each traffic lane is assigned a priority level.
- The lane with the highest priority receives the green signal first.
- Emergency vehicle lanes are immediately selected.
- Remaining lanes are served according to their priority level.

ADVANTAGES:

- Provides immediate access to emergency vehicles.
- Reduces emergency response time.
- Improves public safety.
- Handles high-density traffic more effectively.
- Efficient during peak traffic hours.

DISADVANTAGES:

- Normal traffic lanes may experience longer waiting times.
- Continuous emergency traffic may cause starvation of other lanes.
- More complex to implement than Round Robin.

EXAMPLE:

Consider an intersection with four traffic lanes.

LANE	VEHICLE TYPE	VEHICLE DENSITY	PRIORITY
L1	Normal Traffic	30 Vehicles	3
L2	Ambulance	2 Vehicles	1
L3	Normal Traffic	25 Vehicles	3
L4	Fire Engine	1 Vehicle	1

Priority: 1 = Highest, 3 = Lowest

ROUND ROBIN EXECUTION:

Execution Order:

L1 → L2 → L3 → L4 → Repeat

Each lane receives an equal green signal time of **30 seconds**.

OBSERVATION:

- Every lane gets equal opportunity.
- Ambulance and fire engine must wait for their turn.
- Waiting time for emergency vehicles increases.

PRIORITY SCHEDULING EXECUTION:

Execution Order:

L2 → L4 → L1 → L3

Emergency vehicle lanes receive the green signal immediately before normal traffic lanes.

OBSERVATION:

- Ambulance and fire engine cross the intersection without delay.
- Normal traffic waits slightly longer.
- Emergency response becomes much faster.

COMPARISON:

FEATURE	ROUND ROBIN	PRIORITY SCHEDULING
Scheduling Basis	Equal Time Quantum	Priority Level
Type	Preemptive	Preemptive / Non-Preemptive
Fairness	High	Moderate
Emergency Vehicle Handling	Delayed	Immediate
Waiting Time for Normal Vehicles	Balanced	May Increase
Starvation	No	Possible

FEATURE	ROUND ROBIN	PRIORITY SCHEDULING
Context Switching	Moderate	Low
Best Application	Normal Traffic	Emergency Situations

ANALYSIS:

Round Robin Scheduling is suitable for normal traffic conditions because every lane receives equal green signal time, ensuring fairness and preventing starvation. However, it cannot provide immediate access to emergency vehicles. Priority Scheduling is more suitable for smart city traffic systems because it allows ambulances, fire engines, and police vehicles to cross the intersection without delay. Although normal vehicles may experience a slight increase in waiting time, the overall safety and efficiency of the transportation system improve significantly.
To prevent starvation of normal traffic lanes, the **Aging** technique can be applied. Aging gradually increases the priority of lanes that have been waiting for a long time, ensuring that every lane eventually receives the green signal.

RECOMMENDED SCHEDULING STRATEGY:

A **hybrid scheduling strategy** combining **Priority Scheduling** and **Round Robin** is the most effective solution for smart city traffic management.

- Priority Scheduling should be used whenever emergency vehicles are detected.
- Round Robin Scheduling should be used during normal traffic conditions to provide equal signal time to all lanes.
- Aging can be implemented to prevent starvation of heavily delayed lanes.

This hybrid approach minimizes vehicle waiting time while ensuring fast and safe movement of emergency vehicles.

CONCLUSION:

For a smart city traffic management system, **Priority Scheduling is more effective than Round Robin Scheduling** because it provides immediate access to emergency vehicles and reduces emergency response time. **Round Robin Scheduling** ensures fairness among all traffic lanes during normal conditions. Therefore, combining **Priority Scheduling, Round Robin Scheduling, and Aging** offers the best solution by balancing fairness, efficiency, and public safety.

5.During an online examination, thousands of students submit answers simultaneously. The server processes requests based on arrival times and submission priorities.Construct a process scheduling table for eight requests and calculate completion time, turnaround time, and waiting time using FCFS and SJF scheduling.

INTRODUCTION:

During an online examination, thousands of students submit their answers simultaneously. Each submission request is treated as a **process**, and the examination server acts as the **CPU** that processes these requests. The operating system uses CPU scheduling algorithms to determine the order in which requests are processed. Efficient scheduling helps reduce waiting time, improve server performance, and ensure timely processing of answer submissions. Two commonly used scheduling algorithms for this scenario are **First Come First Serve (FCFS)** and **Shortest Job First (SJF)**.

FIRST COME FIRST SERVE (FCFS):

FCFS is the simplest CPU scheduling algorithm in which requests are processed in the order of their arrival.

WORKING:

- Student submissions are placed in the ready queue according to arrival time.
- The request that arrives first is processed first.
- Once processing begins, the request continues until completion.
- It is a non-preemptive scheduling algorithm.

ADVANTAGES:

- Simple and easy to implement.
- Fair because requests are processed in arrival order.
- No starvation occurs.
- Suitable when all requests require nearly equal processing time.

DISADVANTAGES:

- Long requests delay shorter requests.
- Average waiting time becomes high.
- Server response time may decrease during heavy traffic.
- Suffers from the convoy effect.

SHORTEST JOB FIRST (SJF):

Shortest Job First executes the request having the smallest processing time before longer requests.

WORKING:

- The burst time of each request is compared.
- The request with the shortest burst time is selected first.
- After completion, the next shortest request is processed.
- It can be implemented as preemptive or non-preemptive scheduling.

ADVANTAGES:

- Produces minimum average waiting time.
- Improves turnaround time.
- Increases server throughput.
- Better utilization of CPU resources.

DISADVANTAGES:

- Burst time must be estimated in advance.
- Long requests may experience starvation.

- More complex than FCFS.

EXAMPLE:
Suppose the examination server receives the following eight submission requests.

REQUEST	ARRIVAL TIME (ms)	BURST TIME (ms)
R1	0	6
R2	1	2
R3	2	8
R4	3	3
R5	4	4
R6	5	2
R7	6	1
R8	7	5

FCFS EXECUTION ORDER:
R1 → R2 → R3 → R4 → R5 → R6 → R7 → R8
OBSERVATION:
- Requests are processed according to arrival time.
- Smaller requests such as **R7** must wait behind larger requests.
- Average waiting time increases during peak submission periods.
- Server performance decreases when long requests arrive first.

SJF EXECUTION ORDER:
R7 → R2 → R6 → R4 → R5 → R8 → R1 → R3
OBSERVATION:
- Requests requiring less processing time are completed first.
- Average waiting time is significantly reduced.
- More requests are completed in less time.
- Overall server efficiency improves.

PROCESS SCHEDULING TABLE (EXAMPLE):

REQUEST	ARRIVAL TIME	BURST TIME	COMPLETION TIME	TURNAROUND TIME	WAITING TIME
R1	0	6	6	6	0
R2	1	2	8	7	5
R3	2	8	16	14	6
R4	3	3	19	16	13
R5	4	4	23	19	15
R6	5	2	25	20	18
R7	6	1	26	20	19
R8	7	5	31	24	19

Formulae:

- **Completion Time (CT)** = Time at which the request finishes execution.
- **Turnaround Time (TAT) = CT − Arrival Time**
- **Waiting Time (WT) = TAT − Burst Time**

(Note: The above table is provided as a simple illustration. In an actual numerical problem, CT, TAT, and WT are calculated based on the exact Gantt chart.)

COMPARISON:

FEATURE	**FCFS**	**SJF**
Scheduling Basis	Arrival Time	Shortest Burst Time
Type	Non-Preemptive	Preemptive / Non-Preemptive
Average Waiting Time	High	Low
Turnaround Time	High	Low
Throughput	Moderate	High
Fairness	High	Moderate
Starvation	No	Possible
Best Application	Simple Batch Systems	High-Speed Server Processing

ANALYSIS:

FCFS processes requests strictly according to arrival time, ensuring fairness but increasing waiting time when large requests arrive before smaller ones. This may delay the processing of quick submissions during peak examination periods.
SJF improves overall server performance by processing shorter requests first. This reduces the average waiting time and turnaround time, allowing more student submissions to be processed quickly. However, long requests may experience starvation if short requests continue to arrive.
To prevent starvation in SJF, the operating system can use the **Aging** technique, where the priority of waiting requests gradually increases over time.

CONCLUSION:

For an online examination server, **Shortest Job First (SJF)** is more efficient than **FCFS** because it minimizes waiting time, reduces turnaround time, and improves server throughput. **FCFS** is simple and fair but becomes less efficient during heavy server loads. Therefore, **SJF is the recommended scheduling algorithm** for processing online examination submissions efficiently.

6.A factory assembly line consists of multiple robotic arms, each performing a task. If one robot fails, the operating system must reschedule pending tasks.Analyze how process states (Ready, Running, Waiting, Terminated) change during task execution and explain how scheduling ensures continuous production.

INTRODUCTION:

A factory assembly line consists of multiple robotic arms that perform different manufacturing tasks such as welding, drilling, painting, assembling, and packaging. In an Operating System, each robotic task is considered a **process**, while the processor (CPU) controls and schedules these processes. During production, if one robotic arm fails due to a hardware fault or maintenance issue, the operating system must reschedule the remaining tasks to other available robots. This ensures continuous production, reduces downtime, and improves overall productivity.

PROCESS STATES:

In Operating Systems, every process passes through different states during its execution. These states help the operating system monitor and control task execution efficiently.

READY STATE:

A process enters the Ready state after it is created and is waiting for CPU allocation. In the factory, a robotic task is ready to begin but is waiting for an available robotic arm.

RUNNING STATE:

When the CPU allocates a robotic arm to execute the assigned task, the process enters the Running state. The robot actively performs operations such as welding, painting, or assembling.

WAITING (BLOCKED) STATE:

A process enters the Waiting state when it cannot continue execution because it is waiting for a required resource. For example, if a robot is waiting for raw materials, spare parts, sensor input, or another robot to finish its task, the process remains in the Waiting state until the required resource becomes available.

TERMINATED STATE:

Once the robot successfully completes its assigned task, the process enters the Terminated state. The operating system removes the completed process from memory and allocates resources to the next task.

PROCESS STATE TRANSITION:

The normal sequence of process execution is:

New → Ready → Running → Waiting (if required) → Ready → Running → Terminated

This process cycle continues until all manufacturing tasks are completed successfully.

TASK RESCHEDULING DURING ROBOT FAILURE:

If one robotic arm fails while executing a task, the operating system immediately detects the failure and reschedules the unfinished task to another available robotic arm.

The scheduling process includes:

- Detecting the failed robot.
- Moving the unfinished task from the Running state to the Ready state.
- Selecting another available robot.

- Assigning the pending task to the new robot.
- Continuing production without restarting the entire process.

This mechanism minimizes production delays and improves manufacturing efficiency.

EXAMPLE:

Suppose a factory has four robotic arms performing different tasks.

ROBOT	TASK	INITIAL PROCESS STATE
R1	Welding	Running
R2	Painting	Ready
R3	Drilling	Waiting
R4	Packaging	Ready

During execution, **Robot R1 fails** while performing the welding operation.

STATE CHANGES:

TASK	BEFORE FAILURE	AFTER FAILURE
Welding	Running	Ready (Waiting for another robot)
Painting	Ready	Running
Drilling	Waiting	Ready (after resource becomes available)
Packaging	Ready	Running
Welding (Rescheduled)	Ready	Running on another robot

OBSERVATION:

- The operating system detects the robot failure.
- The unfinished welding task is returned to the Ready state.
- Another available robot is assigned to complete the welding process.
- Other robots continue their assigned tasks without interruption.
- Production continues with minimal downtime.

ROLE OF CPU SCHEDULING:

CPU scheduling plays a major role in maintaining continuous production in an automated factory.

Its functions include:

- Allocating robotic tasks efficiently.
- Reducing CPU idle time.
- Balancing workload among robots.
- Preventing resource conflicts.
- Ensuring faster completion of manufacturing tasks.
- Improving overall production efficiency.
- Handling robot failures through task rescheduling.

ADVANTAGES OF PROCESS SCHEDULING IN FACTORY AUTOMATION:

- Ensures uninterrupted production.
- Reduces machine idle time.
- Improves CPU and resource utilization.
- Balances workload among robotic arms.
- Minimizes production delays.

- Increases manufacturing efficiency.
- Enhances system reliability.

DISADVANTAGES:
- Complex scheduling algorithms increase system complexity.
- Frequent task rescheduling may introduce overhead.
- Hardware failures can still reduce production speed.
- Incorrect scheduling decisions may reduce efficiency.

ANALYSIS:

In a factory environment, continuous production is essential. Whenever a robotic arm fails, the operating system immediately changes the process state of the affected task and assigns it to another available robot. The Ready, Running, Waiting, and Terminated states help the operating system monitor every task throughout its execution. Efficient CPU scheduling minimizes downtime, improves resource utilization, and ensures that production targets are achieved without major interruptions.

CONCLUSION:

The Operating System plays a vital role in factory automation by managing process states and scheduling robotic tasks efficiently. When a robot fails, the operating system reschedules pending tasks to available robots, ensuring continuous production. The proper use of **Ready, Running, Waiting, and Terminated** process states, along with efficient scheduling algorithms, increases productivity, reduces downtime, and improves the overall reliability of the manufacturing system.

7.A video streaming service processes user requests for video buffering. Premium users are assigned higher priority than standard users.Given a set of user requests with arrival and burst times, implement Priority Scheduling and evaluate fairness, starvation, and throughput.

INTRODUCTION:

A video streaming service such as Netflix, YouTube, or Amazon Prime receives thousands of user requests every second for video playback and buffering. In an Operating System, each user request is considered a **process**, and the streaming server acts as the **CPU**. To provide better Quality of Service (QoS), premium users are given higher priority than standard users. The operating system uses **Priority Scheduling** to process requests efficiently, reduce buffering time, and improve user experience.

PRIORITY SCHEDULING:

Priority Scheduling is a CPU scheduling algorithm in which every process is assigned a priority value. The process with the highest priority is executed first. In a video streaming service, premium user requests receive higher priority, while standard user requests receive lower priority.

WORKING:
- Every video request is assigned a priority.
- Premium users are assigned higher priority than standard users.
- The server processes the highest-priority request first.

- If two requests have the same priority, they are processed using FCFS.
- This scheduling may be implemented as preemptive or non-preemptive depending on the streaming system.

EXAMPLE:
Suppose a streaming server receives the following user requests.

REQUEST	USER TYPE	ARRIVAL TIME (ms)	BURST TIME (ms)	PRIORITY
U1	Standard	0	6	3
U2	Premium	1	4	1
U3	Standard	2	5	3
U4	Premium	3	3	1
U5	Standard	4	2	3

Priority: 1 = Highest, 3 = Lowest
EXECUTION ORDER:
U2 → U4 → U1 → U3 → U5
OBSERVATION:

- Premium user requests (U2 and U4) are processed before standard user requests.
- Premium users experience faster buffering and reduced waiting time.
- Standard users may wait longer if premium requests continue to arrive.

FAIRNESS:
Fairness refers to providing all users with an opportunity to access streaming services without unnecessary delays.

- Premium users receive better service because they pay for additional benefits.
- Standard users still receive service but may have longer waiting times.
- If only Priority Scheduling is used, fairness decreases because premium requests are always preferred.

STARVATION:
Starvation occurs when low-priority processes wait indefinitely because high-priority processes continuously enter the system.
In this scenario:

- Standard users may experience starvation if premium requests keep arriving.
- Long waiting times can reduce user satisfaction.
- Server resources become concentrated on high-priority users.

SOLUTION USING AGING:
The **Aging** technique gradually increases the priority of waiting standard user requests over time.
Benefits of Aging:

- Prevents starvation.
- Improves fairness.
- Ensures every user request is eventually processed.
- Balances system performance and customer satisfaction.

THROUGHPUT:
Throughput is the number of requests completed successfully in a given period.
Using Priority Scheduling:

- Premium requests are completed quickly.
- More high-priority requests are processed within a short time.
- Overall throughput remains high because important requests are handled efficiently.
- Streaming quality improves for premium users.

ADVANTAGES OF PRIORITY SCHEDULING:

- Reduces buffering time for premium users.
- Improves Quality of Service (QoS).
- Provides faster response time.
- Efficient utilization of server resources.
- Suitable for subscription-based streaming platforms.
- Enhances customer satisfaction for premium subscribers.

DISADVANTAGES OF PRIORITY SCHEDULING:

- Standard users may experience longer waiting times.
- Starvation may occur for low-priority users.
- Priority assignment increases scheduling complexity.
- Fairness among all users is reduced.

COMPARISON OF USER EXPERIENCE:

FEATURE	PREMIUM USERS	STANDARD USERS
Priority Level	High	Low
Waiting Time	Very Low	Higher
Buffering Delay	Minimal	Moderate
Response Time	Fast	Slower
Service Quality	Excellent	Good
Chance of Starvation	No	Possible

ANALYSIS:

Priority Scheduling is highly effective in a video streaming service because premium subscribers expect uninterrupted streaming and faster buffering. By processing premium requests first, the server improves customer satisfaction and maintains a high level of service quality. However, continuous arrival of premium requests may delay standard users, resulting in starvation. The Aging technique overcomes this problem by gradually increasing the priority of waiting standard users, ensuring that all requests are eventually processed.

CONCLUSION:

Priority Scheduling is the most suitable scheduling algorithm for video streaming services because it provides faster buffering, lower response time, and improved Quality of Service for premium users. Although starvation may occur for standard users, applying the Aging technique improves fairness and guarantees that every user request is processed. Therefore, **Priority Scheduling combined with Aging** provides an efficient, reliable, and fair solution for modern video streaming platforms.

8.A bank processes ATM withdrawals, deposits, and fund transfers concurrently. Each transaction is treated as a process with varying execution times.Using Round Robin Scheduling (time quantum = 4 ms), determine the execution order and calculate average turnaround and waiting times for ten transactions.

INTRODUCTION:

A bank processes multiple customer transactions such as ATM withdrawals, cash deposits, fund transfers, balance inquiries, and bill payments simultaneously. In an Operating System, each transaction is treated as a **process**, while the bank server acts as the **CPU**. Since many customers access banking services at the same time, the operating system uses **Round Robin Scheduling** to ensure that every transaction receives a fair share of CPU time. This improves responsiveness and prevents any transaction from waiting indefinitely.

ROUND ROBIN SCHEDULING:

Round Robin (RR) is a **preemptive CPU scheduling algorithm** in which every process is allocated a fixed amount of CPU time known as the **time quantum**. In this problem, the time quantum is **4 milliseconds (ms)**.

WORKING:

- All transactions are placed in the ready queue.
- The first transaction receives the CPU for a maximum of **4 ms**.
- If the transaction finishes within 4 ms, it is removed from the queue.
- If it is not completed, the remaining execution time is placed at the end of the ready queue.
- The CPU continues executing transactions in a circular manner until all transactions are completed.

EXAMPLE:

Suppose the bank server receives the following ten transactions.

TRANSACTION	TYPE	BURST TIME (ms)
T1	ATM Withdrawal	6
T2	Cash Deposit	3
T3	Fund Transfer	7
T4	Balance Inquiry	2
T5	Bill Payment	5
T6	ATM Withdrawal	4
T7	Cash Deposit	6
T8	Fund Transfer	3
T9	Balance Inquiry	5
T10	Mini Statement	2

Time Quantum = 4 ms

EXECUTION ORDER:

T1 → T2 → T3 → T4 → T5 → T6 → T7 → T8 → T9 → T10 → T1 → T3 → T5 → T7 → T9

OBSERVATION:

- Transactions requiring **4 ms or less** complete in one CPU cycle.

- Transactions requiring more than **4 ms** are placed back into the ready queue with their remaining burst time.
- Every transaction gets an equal opportunity to execute.

AVERAGE TURNAROUND TIME AND WAITING TIME:

For every transaction:

- **Turnaround Time (TAT) = Completion Time − Arrival Time**
- **Waiting Time (WT) = Turnaround Time − Burst Time**

Round Robin scheduling generally provides:

- Moderate average waiting time.
- Moderate turnaround time.
- Good response time.
- Fair CPU allocation among all transactions.

ADVANTAGES OF ROUND ROBIN SCHEDULING:

- Provides equal CPU time to every transaction.
- Prevents starvation of any transaction.
- Improves response time for users.
- Suitable for multi-user banking systems.
- Ensures fairness among all banking operations.
- Supports time-sharing environments.
- Easy to implement.

DISADVANTAGES OF ROUND ROBIN SCHEDULING:

- Frequent context switching increases CPU overhead.
- Performance depends on the selected time quantum.
- A very small time quantum reduces efficiency.
- A very large time quantum behaves like FCFS scheduling.

COMPARISON WITH FCFS:

FEATURE	**ROUND ROBIN**	**FCFS**
Scheduling Basis	Time Quantum	Arrival Time
Type	Preemptive	Non-Preemptive
Fairness	High	Moderate
Response Time	Fast	Slow
Starvation	Not Possible	Not Possible
Context Switching	High	Low
Suitable For	Interactive Banking Systems	Batch Processing

ANALYSIS:

In a banking environment, customers expect quick responses for their transactions. Round Robin Scheduling ensures that every transaction receives CPU time without waiting indefinitely. Transactions such as balance inquiries and mini statements are completed quickly because their burst times are less than the time quantum. Longer transactions, such as fund transfers, continue execution in the next cycle until completion. This approach improves system responsiveness and maintains fairness among all customers.

Although Round Robin introduces additional context switching, it provides better performance for banking applications where many users access the server simultaneously.

CONCLUSION:

Round Robin Scheduling is the most suitable scheduling algorithm for banking systems because it allocates CPU time fairly to every transaction and prevents starvation. Using a **4 ms time quantum**, the operating system efficiently processes withdrawals, deposits, fund transfers, and other banking operations while maintaining good response time and overall system performance. Therefore, Round Robin Scheduling is widely used in multi-user and time-sharing environments such as modern banking systems.

9.An autonomous vehicle executes multiple processes simultaneously: obstacle detection, GPS navigation, speed control, and lane detection.Identify the process states of each task at different time intervals and justify the use of preemptive scheduling in ensuring passenger safety.

INTRODUCTION:

An autonomous (self-driving) vehicle performs multiple tasks simultaneously to ensure safe and efficient driving. These tasks include **obstacle detection, GPS navigation, speed control, and lane detection**. In an Operating System, each task is treated as a **process**, while the vehicle's onboard computer acts as the **CPU**. The operating system manages these processes using **process states** and **preemptive scheduling** to provide quick responses to changing road conditions. Efficient scheduling is essential to ensure passenger safety and reliable vehicle performance.

PROCESS STATES:

A process passes through different states during its execution. The operating system changes the process state based on the availability of CPU and required resources.

NEW STATE:

A process enters the New state when it is created. For example, when the vehicle starts, processes such as GPS navigation, obstacle detection, lane detection, and speed control are initialized.

READY STATE:

After creation, the process enters the Ready state. It waits for CPU allocation before execution. For example, the lane detection process waits until the processor is available.

RUNNING STATE:

When the CPU allocates processing time, the process enters the Running state. During this state, the process performs its assigned task, such as detecting obstacles or calculating the vehicle's route.

WAITING (BLOCKED) STATE:

A process enters the Waiting state when it requires additional resourcesor input. For example, the GPS navigation process waits for updated satellite signals, or the obstacle detection process waits for sensor data.

TERMINATED STATE:

After completing its task successfully, the process enters the Terminated state, and the operating system releases the allocated resources.

PREEMPTIVE SCHEDULING:

Preemptive Scheduling is a CPU scheduling algorithm in which a high-priority process can interrupt a currently running low-priority process whenever immediate execution is required.

WORKING:

- The operating system continuously monitors all running processes.
- If a higher-priority process becomes ready, the CPU immediately switches to it.
- The interrupted process is moved back to the Ready state.
- After the high-priority task finishes, the interrupted process resumes execution.

EXAMPLE:

Suppose an autonomous vehicle is performing the following tasks.

PROCESS	FUNCTION	PRIORITY
P1	GPS Navigation	3
P2	Obstacle Detection	1
P3	Speed Control	2
P4	Lane Detection	2

Priority: 1 = Highest, 3 = Lowest

Initially, the CPU executes **GPS Navigation (P1)**.

Suddenly, an obstacle appears in front of the vehicle.

PROCESS STATE CHANGES:

PROCESS	BEFORE OBSTACLE	AFTER OBSTACLE DETECTED
GPS Navigation (P1)	Running	Ready
Obstacle Detection (P2)	Ready	Running
Speed Control (P3)	Ready	Waiting
Lane Detection (P4)	Ready	Waiting

OBSERVATION:

- GPS Navigation is interrupted immediately.
- Obstacle Detection receives the CPU because it has the highest priority.
- After the obstacle is avoided, GPS Navigation resumes execution.
- This quick switching helps prevent accidents and ensures passenger safety.

IMPORTANCE OF PREEMPTIVE SCHEDULING:

- Provides immediate response to emergency situations.
- Executes safety-critical tasks without delay.
- Improves reaction time.
- Prevents vehicle collisions.
- Ensures smooth coordination between multiple vehicle systems.
- Supports real-time decision-making.

ADVANTAGES OF PREEMPTIVE SCHEDULING:

- Fast response to critical events.
- Efficient handling of multiple tasks.
- Better CPU utilization.
- Improves vehicle safety.
- Suitable for real-time embedded systems.
- Reduces the risk of accidents.
- Ensures reliable vehicle operation.

DISADVANTAGES OF PREEMPTIVE SCHEDULING:

- Frequent context switching increases CPU overhead.
- Scheduling logic is more complex.
- Improper priority assignment may affect performance.
- Requires high processing capability.

ANALYSIS:

An autonomous vehicle continuously receives data from cameras, sensors, GPS, and radar systems. Some tasks, such as obstacle detection and speed control, are safety-critical and require immediate CPU access. Preemptive Scheduling ensures that these high-priority processes interrupt less important tasks whenever necessary. Process states such as Ready, Running, Waiting, and Terminated help the operating system manage each task efficiently. This approach minimizes response time, improves reliability, and enhances passenger safety.

CONCLUSION:

Preemptive Scheduling is the most suitable scheduling algorithm for autonomous vehicles because it allows high-priority safety tasks to interrupt lower-priority processes whenever required. The use of process states helps the operating system manage multiple tasks efficiently while ensuring smooth vehicle operation. Therefore, **Preemptive Scheduling combined with effective process state management** provides high reliability, improved performance, and maximum passenger safety in autonomous vehicles.

10.A satellite control center monitors communication, navigation, power management, and telemetry systems. Communication tasks have the highest priority during emergencies.Simulate process execution using Priority Scheduling and analyze the impact of process starvation. Propose a solution using aging techniques to improve fairness.

INTRODUCTION:

A satellite control center continuously monitors and controls important satellite operations such as **communication, navigation, power management, and telemetry**. Each operation is treated as a **process**, while the satellite control computer acts as the **CPU**. During emergency situations, communication tasks become more important than other tasks because they maintain contact between the satellite and the ground station. To manage these operations efficiently, the operating system uses **Priority Scheduling**. However, continuous execution of high-priority processes may cause **starvation** of low-priority processes. To overcome this problem, the **Aging** technique is used.

PRIORITY SCHEDULING:

Priority Scheduling is a CPU scheduling algorithm in which each process is assigned a priority. The process with the highest priority is executed before the others.

WORKING:

- Every satellite operation is assigned a priority.
- Communication tasks receive the highest priority during emergencies.
- The CPU always selects the highest-priority process.
- If two processes have the same priority, they are executed using FCFS.
- The process continues until all operations are completed.

EXAMPLE:

Suppose the satellite control center has the following processes.

PROCESS	OPERATION	PRIORITY
P1	Communication	1
P2	Navigation	2
P3	Power Management	3
P4	Telemetry	4

Priority: 1 = Highest, 4 = Lowest

EXECUTION ORDER:

P1 → P2 → P3 → P4

During an emergency, suppose another communication process (**P5**) arrives.

PROCESS	OPERATION	PRIORITY
P5	Emergency Communication	1

The new execution order becomes:

P1 → P5 → P2 → P3 → P4

OBSERVATION:

- Communication tasks are always executed first.
- Emergency messages are transmitted without delay.
- Navigation and power management wait until communication tasks finish.
- Telemetry receives the lowest priority and waits the longest.

STARVATION:

Starvation occurs when low-priority processes remain in the ready queue for a long time because high-priority processes continuously receive CPU time.

In this scenario:

- Communication tasks continue arriving during emergencies.
- Telemetry processes remain waiting for a long period.
- Important monitoring information may be delayed.
- Overall system fairness decreases.

AGING TECHNIQUE:

Aging is a technique used to prevent starvation by gradually increasing the priority of waiting processes.

WORKING OF AGING:

- Every process starts with its assigned priority.
- If a process waits for a long time, its priority is gradually increased.
- Eventually, the waiting process becomes high enough to receive CPU time.

- This ensures that every process is executed without indefinite waiting.

EXAMPLE OF AGING:

Initially:

PROCESS	PRIORITY
Communication	1
Navigation	2
Power Management	3
Telemetry	4

After waiting for a long time:

PROCESS	UPDATED PRIORITY
Communication	1
Navigation	2
Power Management	2
Telemetry	1

Now, the Telemetry process receives CPU time and starvation is eliminated.

ADVANTAGES OF PRIORITY SCHEDULING:

- Immediate execution of critical communication tasks.
- Fast response during emergencies.
- Efficient utilization of CPU resources.
- Suitable for real-time satellite operations.
- Improves mission reliability.
- Ensures quick handling of high-priority events.

DISADVANTAGES OF PRIORITY SCHEDULING:

- Low-priority processes may suffer starvation.
- Fairness among processes decreases.
- Requires proper priority assignment.
- Scheduling becomes more complex.

ADVANTAGES OF AGING:

- Prevents starvation.
- Improves fairness among all processes.
- Ensures every process eventually gets CPU time.
- Balances system performance.
- Increases reliability of satellite operations.

COMPARISON:

FEATURE	PRIORITY SCHEDULING	AGING TECHNIQUE
Scheduling Basis	Priority Level	Waiting Time
Emergency Response	Very Fast	Fast with Fairness
Starvation	Possible	Eliminated
Fairness	Moderate	High
System Reliability	High	Very High
Suitable For	Critical Operations	Long-Running Systems

ANALYSIS:
Priority Scheduling is highly effective for satellite control because communication tasks are critical during emergencies and must be executed immediately. However, continuous arrival of high-priority communication processes may delay navigation, power management, and telemetry tasks. The Aging technique solves this problem by gradually increasing the priority of waiting processes, ensuring that every operation receives CPU time. This improves both fairness and system reliability without affecting emergency communication.

CONCLUSION:
Priority Scheduling is the most suitable scheduling algorithm for a satellite control center because it ensures immediate execution of communication tasks during emergencies. However, starvation of low-priority processes such as telemetry may occur. By applying the **Aging technique**, the operating system gradually increases the priority of waiting processes, eliminating starvation and improving fairness. Therefore, **Priority Scheduling combined with Aging** provides an efficient, reliable, and fair scheduling solution for satellite control systems.

11.A set of processes arrive at time 0 with the following burst times:

Process

Burst Time (ms)

P1

10

P2

4

P3

6

P4

2
a. Calculate Waiting Time and Turnaround Time for each process using SJF (Non-preemptive).
b. Compute the average waiting time and average turnaround time.
c. Interpret whether SJF improves CPU utilization compared to FCFS for this workload.

(a) Calculate Waiting Time and Turnaround Time using SJF (Non-Preemptive)
Step 1: Arrange the processes according to burst time (Shortest Job First)
Process Burst Time (ms)

Process	Burst Time (ms)
P4	2
P2	4
P3	6
P1	10

Step 2: Draw the Gantt Chart

0 2 6 12 22

| P4 | P2 | P3 | P1 |

Step 3: Calculate Waiting Time (WT)

Formula:

Waiting Time (WT) = Start Time − Arrival Time

Since all processes arrive at time **0**:

Process	Arrival Time	Start Time	Waiting Time (WT)
P4	0	0	0
P2	0	2	2
P3	0	6	6
P1	0	12	12

Step 4: Calculate Turnaround Time (TAT)

Formula:

Turnaround Time (TAT) = Completion Time − Arrival Time

Process	Completion Time (CT)	Arrival Time (AT)	Turnaround Time (TAT)
P4	2	0	2
P2	6	0	6
P3	12	0	12
P1	22	0	22

Final Table

Process	Burst Time	Waiting Time (WT)	Turnaround Time (TAT)
P4	2	0	2
P2	4	2	6
P3	6	6	12
P1	10	12	22

(b) Calculate the Average Waiting Time and Average Turnaround Time

Average Waiting Time (AWT)

$$\text{AWT} = \frac{0 + 2 + 6 + 12}{4} = \frac{20}{4} = 5 \text{ ms}$$

Average Waiting Time = 5 ms

Average Turnaround Time (ATAT)

$$\text{ATAT} = \frac{2 + 6 + 12 + 22}{4} = \frac{42}{4} = 10.5 \text{ ms}$$

Average Turnaround Time = 10.5 ms

(c) Interpretation: Does SJF improve CPU utilization compared to FCFS?
Answer:
Yes, **Shortest Job First (SJF)** improves overall system performance compared to **First Come First Serve (FCFS)** for this workload.
Reasons:

- SJF executes the shortest processes first, reducing the average waiting time.
- Short processes complete earlier, improving turnaround time.
- More processes are completed in a shorter period, increasing throughput.
- CPU remains busy executing tasks efficiently with less idle time.
- User response time is better because smaller jobs finish quickly.

In **FCFS**, if a long process executes first, shorter processes must wait, increasing the average waiting time. This is known as the **Convoy Effect**.
Therefore, **SJF provides better CPU utilization and overall system performance** than FCFS for this set of processes.

Conclusion
Using **SJF (Non-Preemptive)** scheduling:

- **Execution Order:** P4 → P2 → P3 → P1
- **Average Waiting Time: 5 ms**
- **Average Turnaround Time: 10.5 ms**

SJF is more efficient than FCFS because it minimizes waiting time, improves turnaround time, increases throughput, and provides better CPU utilization. This makes it a suitable scheduling algorithm for workloads where burst times are known in advance.

12. Round Robin Scheduling – Time Quantum Impact
Consider the following processes:

Process

Burst Time (ms)

P1

20

P2

5

P3

13

P4

8
Let the time quantum = 4 ms.
a. Construct the Gantt chart for Round Robin.
b. Calculate Waiting Time and Turnaround Time of each process.
c. Analyze how changing the quantum to 10 ms would affect system performance (CPU utilization & context switching).

(a) Construct the Gantt Chart
Step 1: Execute using Round Robin (TQ = 4 ms)

- P1 executes for 4 ms → Remaining = 16 ms
- P2 executes for 4 ms → Remaining = 1 ms
- P3 executes for 4 ms → Remaining = 9 ms
- P4 executes for 4 ms → Remaining = 4 ms
- P1 executes for 4 ms → Remaining = 12 ms
- P2 executes for 1 ms → Completed
- P3 executes for 4 ms → Remaining = 5 ms
- P4 executes for 4 ms → Completed
- P1 executes for 4 ms → Remaining = 8 ms
- P3 executes for 4 ms → Remaining = 1 ms
- P1 executes for 4 ms → Remaining = 4 ms
- P3 executes for 1 ms → Completed
- P1 executes for 4 ms → Completed

Gantt Chart
0 4 8 12 16 20 21 25 29 33 37 38 42
| P1 | P2 | P3 | P4 | P1 | P2 | P3 | P4 | P1 | P3 | P1 | P3 | P1 |

(b) Calculate Waiting Time and Turnaround Time
Step 1: Completion Time (CT)

Process	Completion Time (CT)
P1	42
P2	21
P3	38
P4	29

Step 2: Turnaround Time (TAT)
Formula:
TAT = CT − AT
(All Arrival Times = 0)

Process	CT	AT	TAT
P1	42	0	42
P2	21	0	21
P3	38	0	38
P4	29	0	29

Step 3: Waiting Time (WT)
Formula:
WT = TAT − Burst Time

Process	TAT	Burst Time	Waiting Time (WT)
P1	42	20	22
P2	21	5	16
P3	38	13	25
P4	29	8	21

Final Table

Process	Burst Time	Completion Time	Turnaround Time	Waiting Time
P1	20	42	42	22
P2	5	21	21	16
P3	13	38	38	25
P4	8	29	29	21

Average Waiting Time (AWT)

$$\frac{22 + 16 + 25 + 21}{4} = \frac{84}{4} = 21 \text{ ms}$$

Average Waiting Time = 21 ms

Average Turnaround Time (ATAT)

$$\frac{42 + 21 + 38 + 29}{4} = \frac{130}{4} = 32.5 \text{ ms}$$

Average Turnaround Time = 32.5 ms

(c) Analyze the Effect of Changing the Time Quantum to 10 ms
When the **time quantum is increased from 4 ms to 10 ms**, the performance of the Round Robin scheduling algorithm changes significantly.
Impact on CPU Utilization

- A larger time quantum reduces the number of context switches.
- Less CPU time is wasted switching between processes.
- CPU utilization improves because more time is spent executing processes instead of switching.

Impact on Context Switching

- Context switching decreases considerably.
- Lower overhead improves overall system efficiency.
- System performance becomes faster for longer processes.

Impact on Waiting Time

- Longer processes receive more CPU time in one turn.
- Some shorter processes may wait longer before getting CPU access.
- Average waiting time may increase for short processes.

Impact on Response Time

- Response time for interactive users becomes slightly slower.
- Processes wait longer before receiving their next CPU allocation.

Comparison Table

Parameter	Time Quantum = 4 ms	Time Quantum = 10 ms
Context Switching	High	Low
CPU Utilization	Moderate	High
Response Time	Better	Slightly Lower
Waiting Time	Moderate	May Increase
Throughput	Good	Better
Fairness	High	Moderate

Conclusion

For **Time Quantum = 4 ms**:

- **Execution Order:** P1 → P2 → P3 → P4 → P1 → P2 → P3 → P4 → P1 → P3 → P1 → P3 → P1
- **Average Waiting Time = 21 ms**
- **Average Turnaround Time = 32.5 ms**

Increasing the **time quantum to 10 ms** reduces context switching and improves CPU utilization, but it also reduces responsiveness for short processes. Therefore, selecting an appropriate time quantum is essential to achieve a balance between **CPU utilization, response time, and fairness** in Round Robin Scheduling.

13. CPU Utilization – Multiprogramming Level Analysis

A system has the following process behavior:

- Each process spends 20% time in CPU and 80% waiting for I/O.
- Degree of multiprogramming = 4 processes

CPU Utilization formula:

U=1-(pn)

where:

- p = probability a process is waiting for I/O,
- n = number of processes in memory

a. Calculate CPU Utilization.

b. Compute CPU utilization if the degree of multiprogramming increases to 6.

c. Interpret which configuration improves system performance and why.

Given Data:

- Probability that a process is waiting for I/O, **p = 0.8**
- CPU execution time = **20%**
- I/O waiting time = **80%**
- Degree of Multiprogramming:
  - **Case 1:** $n = 4$
  - **Case 2:** $n = 6$

Formula:

$U = 1 - p^n$

Where:

- **U** = CPU Utilization
- **p** = Probability that a process is waiting for I/O
- **n** = Degree of multiprogramming

(a) Calculate CPU Utilization for Degree of Multiprogramming = 4

Given:

- $p = 0.8$
- $n = 4$

Calculation:

$U = 1 - (0.8)^4$
$U = 1 - 0.4096$
$U = 0.5904$

CPU Utilization

$U = 59.04\%$

Answer: CPU Utilization = 59.04%

(b) Calculate CPU Utilization if Degree of Multiprogramming = 6

Given:

- $p = 0.8$
- $n = 6$

Calculation:

$U = 1 - (0.8)^6$
$U = 1 - 0.262144$
$U = 0.737856$

CPU Utilization

$U = 73.79\%$

Answer: CPU Utilization = 73.79%

(c) Interpretation

The degree of multiprogramming refers to the number of processes present in the main memory at the same time. When one process is waiting for I/O operations, the CPU can execute another ready process. As the number of processes in memory increases, the chances of the CPU remaining idle decrease.

Analysis:

- For **4 processes**, the CPU utilization is **59.04%**.
- For **6 processes**, the CPU utilization increases to **73.79%**.
- Increasing the degree of multiprogramming allows the CPU to remain busy by switching to another process whenever one process waits for I/O.
- This improves overall system performance, increases throughput, and reduces CPU idle time.
- However, if the degree of multiprogramming becomes very high, excessive context switching and memory usage may reduce system performance.

Comparison Table

Degree of Multiprogramming (n)	CPU Utilization (%)
4	59.04%
6	73.79%

Advantages of Higher Multiprogramming

- Improves CPU utilization.

- Reduces CPU idle time.
- Increases system throughput.
- Allows better utilization of system resources.
- Improves overall operating system performance.

Disadvantages of Excessive Multiprogramming

- Increases context switching overhead.
- Requires more main memory.
- Can reduce performance due to excessive process management.
- May lead to resource contention if too many processes compete for the CPU.

Conclusion

Using the formula $U = 1 - p^n$:

- **CPU Utilization for 4 processes = 59.04%**
- **CPU Utilization for 6 processes = 73.79%**

The configuration with **6 processes provides better system performance** because the CPU remains busy for a longer period, reducing idle time and increasing throughput. Therefore, increasing the degree of multiprogramming improves CPU utilization, provided that the system has sufficient memory and resources to manage the additional processes efficiently.